

Esercitazione – Windows Firewall, InetSim e Wireshark

Obiettivo

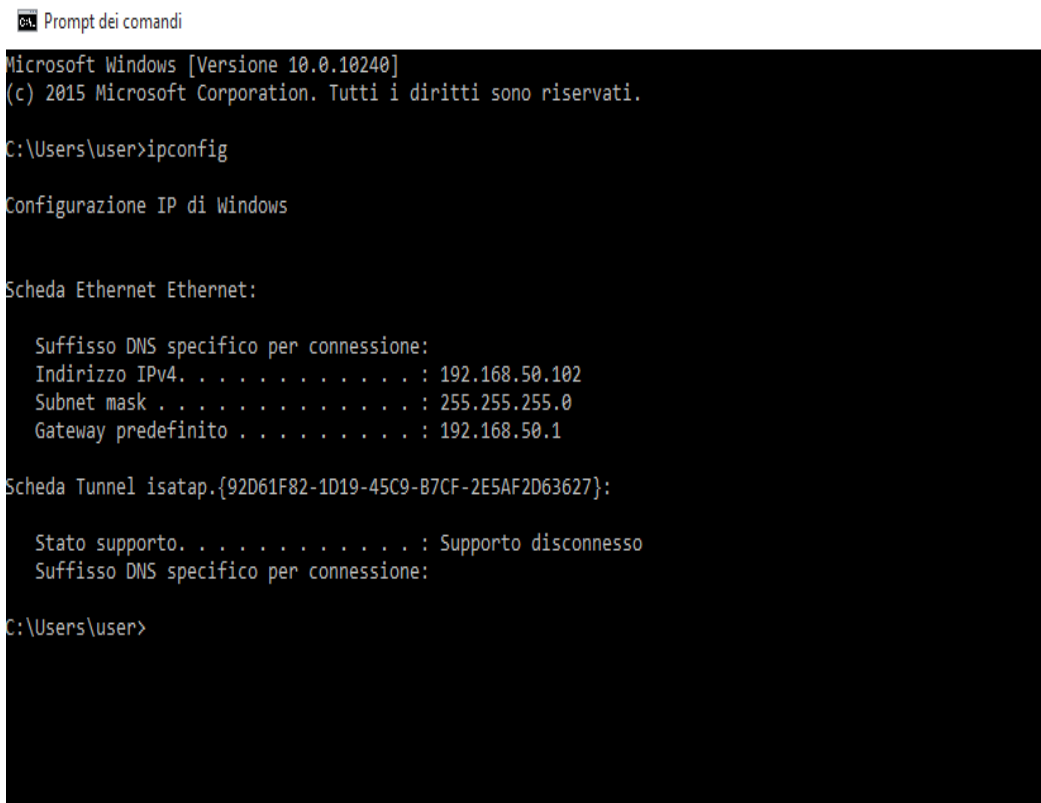
L'obiettivo dell'esercitazione è stato quello di verificare il funzionamento del firewall di Windows, simulare alcuni servizi di rete tramite InetSim e osservare il traffico generato utilizzando Wireshark.

1. Verifica della connettività tra Kali Linux e Windows

Come prima operazione ho controllato che le due macchine virtuali fossero collegate alla stessa rete interna di VirtualBox.

Successivamente ho verificato gli indirizzi IP assegnati alle due macchine.

Figura 1 – Indirizzo IP della macchina Windows

A screenshot of a Windows Command Prompt window. The title bar reads "Prompt dei comandi". The window content shows the output of the 'ipconfig' command. It displays information for the Ethernet adapter and the Tunnel adapter isatap. The Ethernet adapter has an IPv4 address of 192.168.50.102, a subnet mask of 255.255.255.0, and a default gateway of 192.168.50.1. The Tunnel adapter isatap is disconnected.

```
ca Prompt dei comandi
Microsoft Windows [Versione 10.0.10240]
(c) 2015 Microsoft Corporation. Tutti i diritti sono riservati.

C:\Users\user>ipconfig

Configurazione IP di Windows

Scheda Ethernet Ethernet:

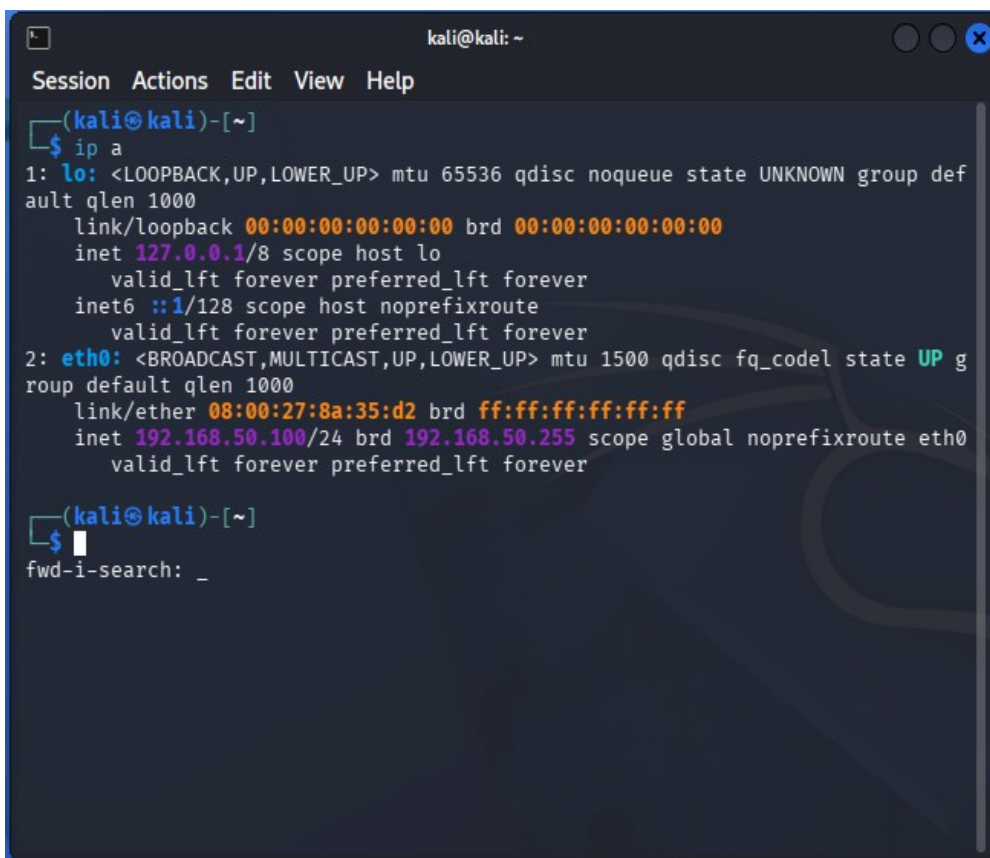
    Suffisso DNS specifico per connessione:
    Indirizzo IPv4. . . . . : 192.168.50.102
    Subnet mask . . . . . : 255.255.255.0
    Gateway predefinito . . . . . : 192.168.50.1

Scheda Tunnel isatap.{92D61F82-1D19-45C9-B7CF-2E5AF2D63627}:

    Stato supporto. . . . . : Supporto disconnesso
    Suffisso DNS specifico per connessione:

C:\Users\user>
```

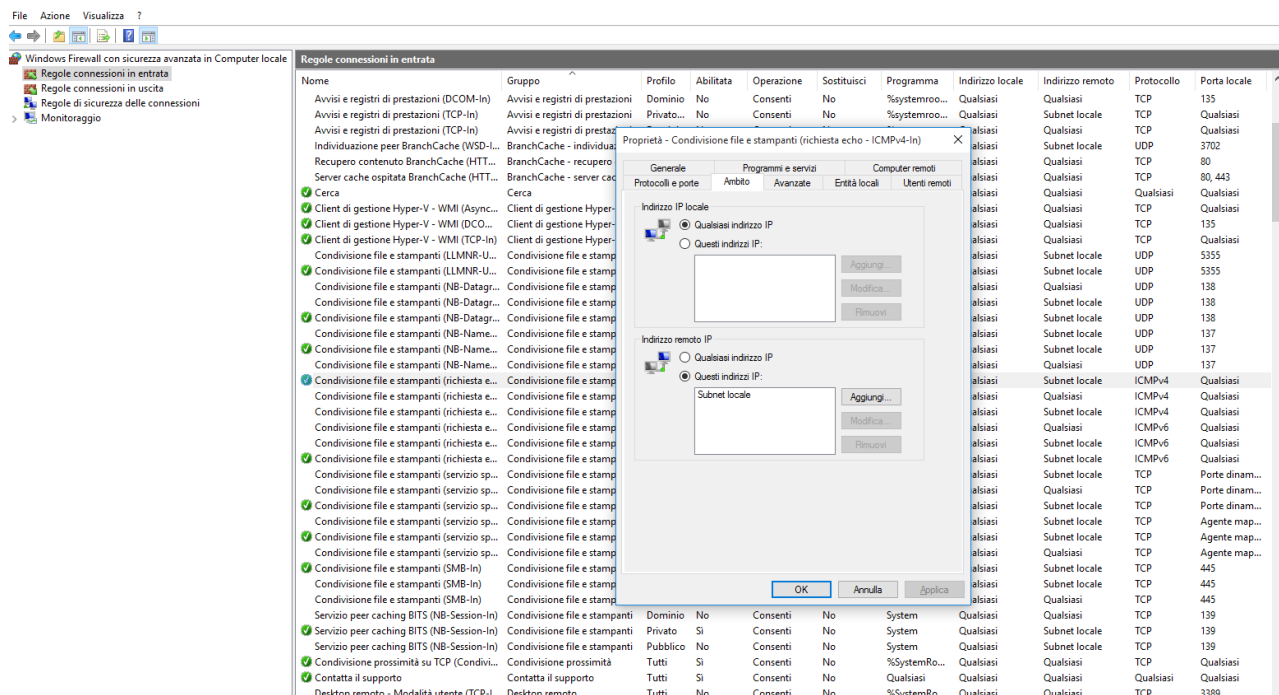
Figura 2 – Indirizzo IP della macchina Kali Linux



```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000  
    link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff  
    inet 192.168.50.100/24 brd 192.168.50.255 scope global noprefixroute eth0  
        valid_lft forever preferred_lft forever  
  
(kali@kali)-[~]  
$  
fwd-i-search: _
```

Le due macchine risultano configurate nella stessa rete:

- Kali Linux: **192.168.50.100**
- Windows: **192.168.50.102**



Dopo aver verificato la configurazione ho eseguito il test di connettività dalla macchina Kali.

Figura 5 – Ping da Kali verso Windows

```
(kali@kali) [~]
$ ping 192.168.50.102
PING 192.168.50.102 (192.168.50.102) 56(84) bytes of data.
64 bytes from 192.168.50.102: icmp_seq=1 ttl=128 time=1.20 ms
64 bytes from 192.168.50.102: icmp_seq=2 ttl=128 time=1.12 ms
64 bytes from 192.168.50.102: icmp_seq=3 ttl=128 time=1.83 ms
64 bytes from 192.168.50.102: icmp_seq=4 ttl=128 time=1.08 ms
64 bytes from 192.168.50.102: icmp_seq=5 ttl=128 time=1.35 ms
64 bytes from 192.168.50.102: icmp_seq=6 ttl=128 time=1.52 ms
64 bytes from 192.168.50.102: icmp_seq=7 ttl=128 time=1.58 ms
64 bytes from 192.168.50.102: icmp_seq=8 ttl=128 time=1.11 ms
64 bytes from 192.168.50.102: icmp_seq=9 ttl=128 time=1.26 ms
64 bytes from 192.168.50.102: icmp_seq=10 ttl=128 time=1.05 ms
64 bytes from 192.168.50.102: icmp_seq=11 ttl=128 time=1.58 ms
64 bytes from 192.168.50.102: icmp_seq=12 ttl=128 time=1.71 ms
64 bytes from 192.168.50.102: icmp_seq=13 ttl=128 time=1.46 ms
64 bytes from 192.168.50.102: icmp_seq=14 ttl=128 time=1.30 ms
64 bytes from 192.168.50.102: icmp_seq=15 ttl=128 time=1.60 ms
64 bytes from 192.168.50.102: icmp_seq=16 ttl=128 time=1.15 ms
^C
— 192.168.50.102 ping statistics —
16 packets transmitted, 16 received, 0% packet loss, time 15026ms
rtt min/avg/max/mdev = 1.047/1.367/1.830/0.238 ms

(kali@kali)-[~]
$
fwd-i-search: _
```

Il ping è stato completato correttamente senza perdita di pacchetti, confermando che la comunicazione tra le due macchine era funzionante.

3. Configurazione di InetSim

In seguito ho verificato che InetSim fosse installato sulla macchina Kali.

Figura 6 – Verifica dell'installazione di InetSim

```
(kali㉿kali)-[~]  
$ inetsim --version  
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
```

Ho poi modificato il file di configurazione /etc/inetsim/inetsim.conf, impostando l'indirizzo IP della macchina Kali come indirizzo di ascolto dei servizi simulati e come indirizzo restituito dal server DNS.

Sono state aggiunte le seguenti direttive:

- service_bind_address 192.168.50.100
- dns_default_ip 192.168.50.100

Figura 7 – Configurazione di InetSim

```
#####  
# service_bind_address  
#  
# IP address to bind services to  
#  
# Syntax: service_bind_address <IP address>  
#  
# Default: 127.0.0.1  
#  
#service_bind_address 10.10.10.1  
service_bind_address 192.168.50.100
```

```
#####  
# dns_default_ip  
#  
# Default IP address to return with DNS replies  
#  
# Syntax: dns_default_ip <IP address>  
#  
# Default: 127.0.0.1  
#  
#dns_default_ip 10.10.10.1  
dns_default_ip 192.168.50.100
```

Terminata la modifica ho avviato InetSim con il comando: “**sudo inetSim**” .

Figura 8 – Avvio di InetSim

```
Parsing configuration file.
Configuration file parsed successfully.
== INetSim main process started (PID 38144) ==
Session ID: 38144
Listening on: 192.168.50.100
Real Date/Time: 2026-07-17 04:56:15
Fake Date/Time: 2026-07-17 04:56:15 (Delta: 0 seconds)
Forking services ...
* dns_53_tcp_udp - started (PID 38154)
Attempt to start Net::DNS::Nameserver in a subprocess at /usr/share/perl5/INETSim/DNS.pm line 69.
* irc_6667_tcp - started (PID 38164)
* tftp_69_udp - started (PID 38163)
* ntp_123_udp - started (PID 38165)
* discard_9_tcp - started (PID 38175)
* time_37_udp - started (PID 38170)
* echo_7_tcp - started (PID 38173)
* daytime_13_udp - started (PID 38172)
* smtp_25_tcp - started (PID 38157)
* daytime_13_tcp - started (PID 38171)
* time_37_tcp - started (PID 38169)
* finger_79_tcp - started (PID 38166)
* https_443_tcp - started (PID 38156)
* http_80_tcp - started (PID 38155)
* chargen_19_udp - started (PID 38180)
* ftp_21_tcp - started (PID 38161)
* echo_7_udp - started (PID 38174)
* smtps_465_tcp - started (PID 38158)
* pop3s_995_tcp - started (PID 38160)
* ftps_990_tcp - started (PID 38162)
* ident_113_tcp - started (PID 38167)
* discard_9_udp - started (PID 38176)
* pop3_110_tcp - started (PID 38159)
* quotd_17_tcp - started (PID 38177)
* syslog_514_udp - started (PID 38168)
* dummy_1_tcp - started (PID 38181)
* dummy_1_udp - started (PID 38182)
* chargen_19_tcp - started (PID 38179)
* quotd_17_udp - started (PID 38178)
done.
Simulation running.
```

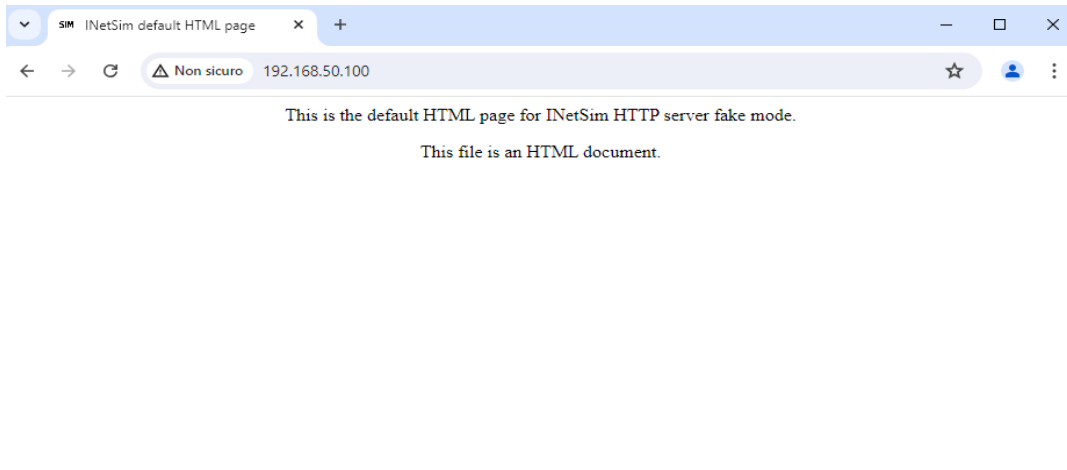
L'avvio è avvenuto correttamente. La presenza della dicitura "**Simulation running**" conferma che InetSim è stato avviato e che i principali servizi simulati sono attivi.

4. Test del server HTTP simulato

Dalla macchina Windows ho aperto il browser e mi sono collegato all'indirizzo: <http://192.168.50.100> .

La pagina HTML predefinita di InetSim è stata visualizzata correttamente.

Figura 9 – Pagina HTTP fornita da InetSim



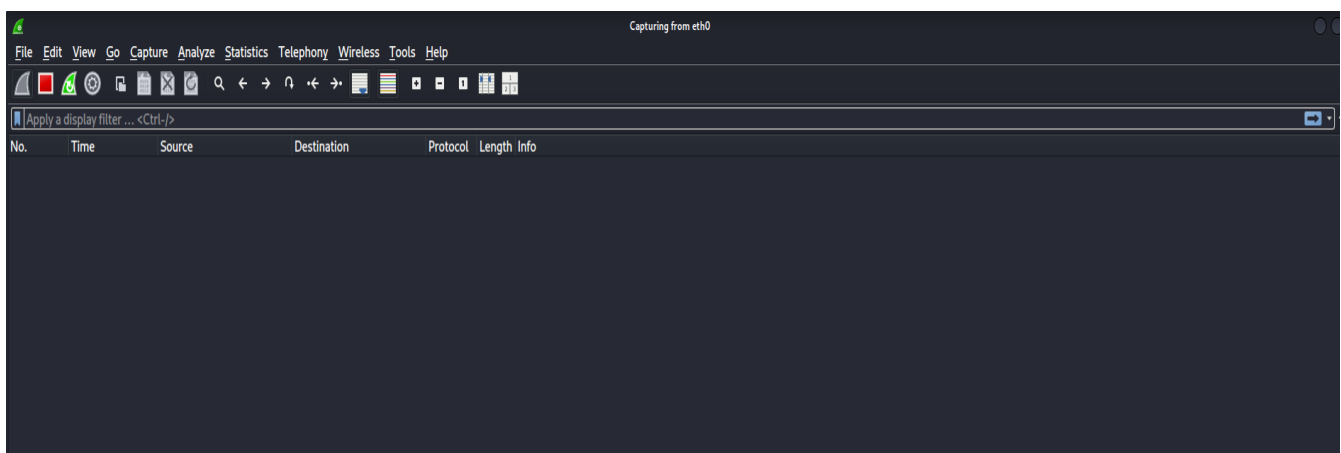
Questo conferma che il server HTTP simulato era correttamente raggiungibile dalla rete.

5. Analisi del traffico con Wireshark

Per osservare la comunicazione tra le due macchine ho avviato Wireshark sulla macchina Kali effettuando la cattura sull'interfaccia di rete “eth0”.

Durante il collegamento al server HTTP sono stati catturati diversi pacchetti.

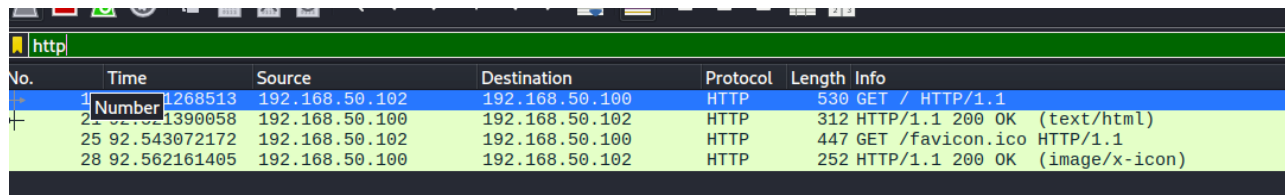
Figura 10 – Wireshark



Traffico HTTP

Applicando il filtro “http” sono risultate visibili le richieste HTTP inviate dal browser e le relative risposte del server InetSim.

Figura 11 – Traffico HTTP



No.	Time	Source	Destination	Protocol	Length	Info
1	12.68513	192.168.50.102	192.168.50.100	HTTP	530	GET / HTTP/1.1
2	13.90058	192.168.50.100	192.168.50.102	HTTP	312	HTTP/1.1 200 OK (text/html)
25	92.543072172	192.168.50.102	192.168.50.100	HTTP	447	GET /favicon.ico HTTP/1.1
28	92.562161405	192.168.50.100	192.168.50.102	HTTP	252	HTTP/1.1 200 OK (image/x-icon)

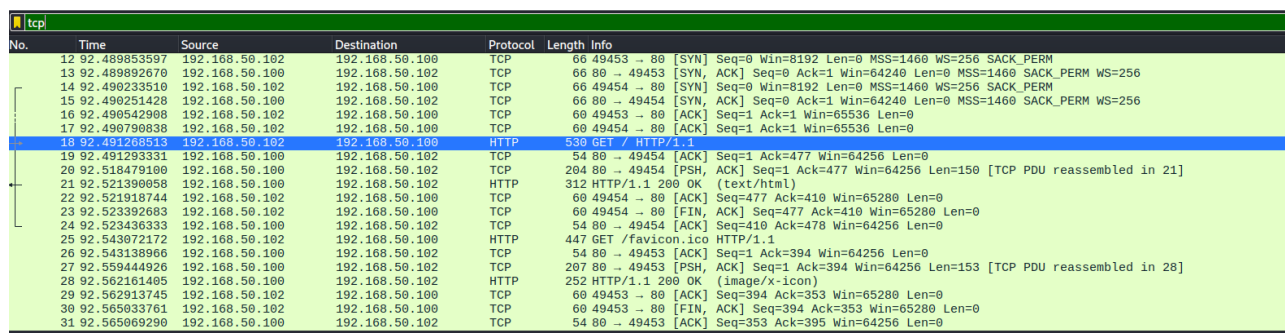
Nella cattura è possibile osservare:

- richiesta **GET /**;
- risposta **HTTP/1.1 200 OK**;
- richiesta del file **favicon.ico**;
- risposta del server.

Traffico TCP

Applicando il filtro “tcp” è stato possibile osservare l'apertura della connessione TCP tra Windows e Kali.

Figura 12 – Traffico TCP



No.	Time	Source	Destination	Protocol	Length	Info
12	92.489853597	192.168.50.102	192.168.50.100	TCP	60	49453 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM
13	92.489892670	192.168.50.100	192.168.50.102	TCP	60	80 → 49453 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=256
14	92.490233510	192.168.50.102	192.168.50.100	TCP	60	49454 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PERM
15	92.490251428	192.168.50.100	192.168.50.102	TCP	60	80 → 49454 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM WS=256
16	92.490542908	192.168.50.102	192.168.50.100	TCP	60	49453 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0
17	92.490790838	192.168.50.100	192.168.50.102	TCP	60	49454 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0
18	92.491268513	192.168.50.102	192.168.50.100	HTTP	530	GET / HTTP/1.1
19	92.491293331	192.168.50.100	192.168.50.102	TCP	54	80 → 49454 [ACK] Seq=1 Ack=477 Win=64256 Len=0
20	92.518479190	192.168.50.100	192.168.50.102	TCP	204	80 → 49454 [PSH, ACK] Seq=1 Ack=477 Win=64256 Len=150 [TCP PDU reassembled in 21]
21	92.521390058	192.168.50.100	192.168.50.102	HTTP	312	HTTP/1.1 200 OK (text/html)
22	92.521918744	192.168.50.102	192.168.50.100	TCP	60	49454 → 80 [ACK] Seq=477 Ack=410 Win=65280 Len=0
23	92.523392683	192.168.50.102	192.168.50.100	TCP	60	49454 → 80 [FIN, ACK] Seq=477 Ack=410 Win=65280 Len=0
24	92.523436333	192.168.50.100	192.168.50.102	TCP	54	80 → 49454 [ACK] Seq=410 Ack=478 Win=64256 Len=0
25	92.543072172	192.168.50.102	192.168.50.100	HTTP	447	GET /favicon.ico HTTP/1.1
26	92.543138966	192.168.50.100	192.168.50.102	TCP	54	80 → 49453 [ACK] Seq=1 Ack=394 Win=64256 Len=0
27	92.559444926	192.168.50.100	192.168.50.102	TCP	207	80 → 49453 [PSH, ACK] Seq=1 Ack=394 Win=64256 Len=153 [TCP PDU reassembled in 28]
28	92.562161405	192.168.50.100	192.168.50.102	HTTP	252	HTTP/1.1 200 OK (image/x-icon)
29	92.562913745	192.168.50.102	192.168.50.100	TCP	60	49453 → 80 [ACK] Seq=394 Ack=353 Win=65280 Len=0
30	92.565033761	192.168.50.102	192.168.50.100	TCP	60	49453 → 80 [FIN, ACK] Seq=394 Ack=353 Win=65280 Len=0
31	92.565069290	192.168.50.100	192.168.50.102	TCP	54	80 → 49453 [ACK] Seq=353 Ack=395 Win=64256 Len=0

Sono visibili le principali fasi della connessione:

- **SYN**
- **SYN/ACK**
- **ACK**

seguite dalla trasmissione dei dati HTTP.

Traffico ARP

Infine ho analizzato il traffico ARP utilizzando il filtro “arp”.

Figura 13 – Traffico ARP

No.	Time	Source	Destination	Protocol	Length	Info
32	96.960167271	PCSSystemtec_id:cd:...	Broadcast	ARP	60	Who has 192.168.50.1? Tell 192.168.50.102
33	97.523132585	PCSSystemtec_id:cd:...	Broadcast	ARP	60	Who has 192.168.50.1? Tell 192.168.50.102
34	98.522352969	PCSSystemtec_id:cd:...	Broadcast	ARP	60	Who has 192.168.50.1? Tell 192.168.50.102
46	286.122881820	PCSSystemtec_id:cd:...	Broadcast	ARP	60	Who has 192.168.50.1? Tell 192.168.50.102
47	287.023243367	PCSSystemtec_id:cd:...	Broadcast	ARP	60	Who has 192.168.50.1? Tell 192.168.50.102
48	288.023326821	PCSSystemtec_id:cd:...	Broadcast	ARP	60	Who has 192.168.50.1? Tell 192.168.50.102

Le richieste ARP vengono utilizzate per risolvere l'indirizzo MAC corrispondente ad un indirizzo IP presente nella rete locale.

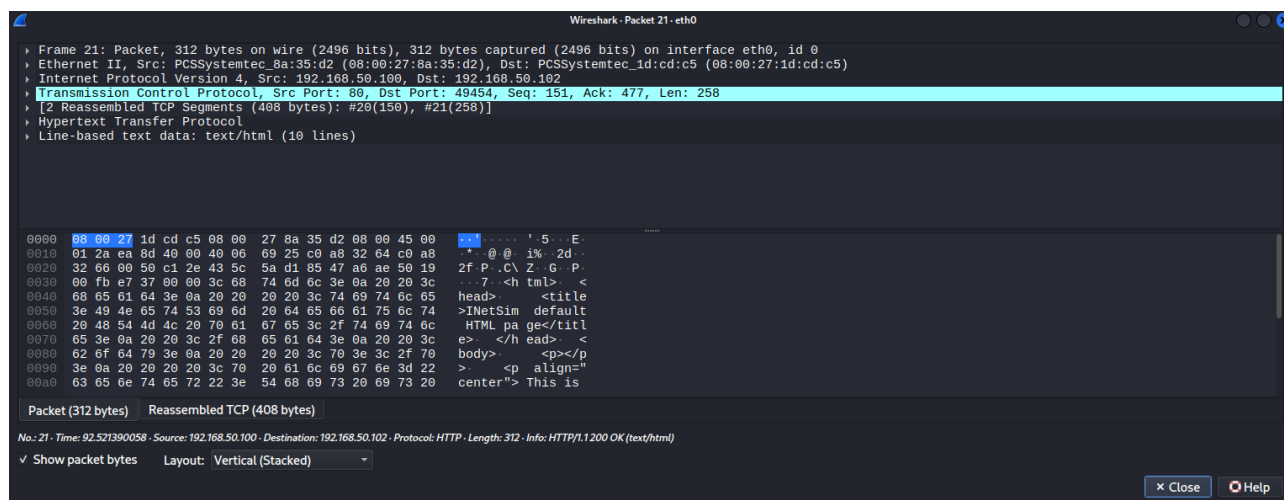
6. Analisi del contenuto dei pacchetti (Attività facoltativa)

Per approfondire il funzionamento della comunicazione di rete ho analizzato il contenuto di uno dei pacchetti HTTP catturati con Wireshark.

Selezionando un pacchetto è stato possibile visualizzare le informazioni relative ai diversi livelli della comunicazione, come l'intestazione Ethernet, il protocollo IP, il protocollo TCP e il protocollo HTTP.

Nel riquadro inferiore di Wireshark è inoltre possibile osservare il contenuto del pacchetto in formato esadecimale e la relativa interpretazione in formato testuale.

Figura 14 – Analisi del contenuto di un pacchetto HTTP



Dall'analisi è possibile osservare:

- l'indirizzo IP sorgente **192.168.50.100**;
- l'indirizzo IP di destinazione **192.168.50.102**;
- il protocollo di trasporto **TCP**;
- la risposta del server **HTTP/1.1 200 OK**;
- il contenuto della pagina HTML restituita da InetSim.

Questa analisi mi ha permesso di comprendere come le informazioni vengano incapsulate nei diversi livelli della comunicazione e come Wireshark consenta di visualizzarne il contenuto nel dettaglio.

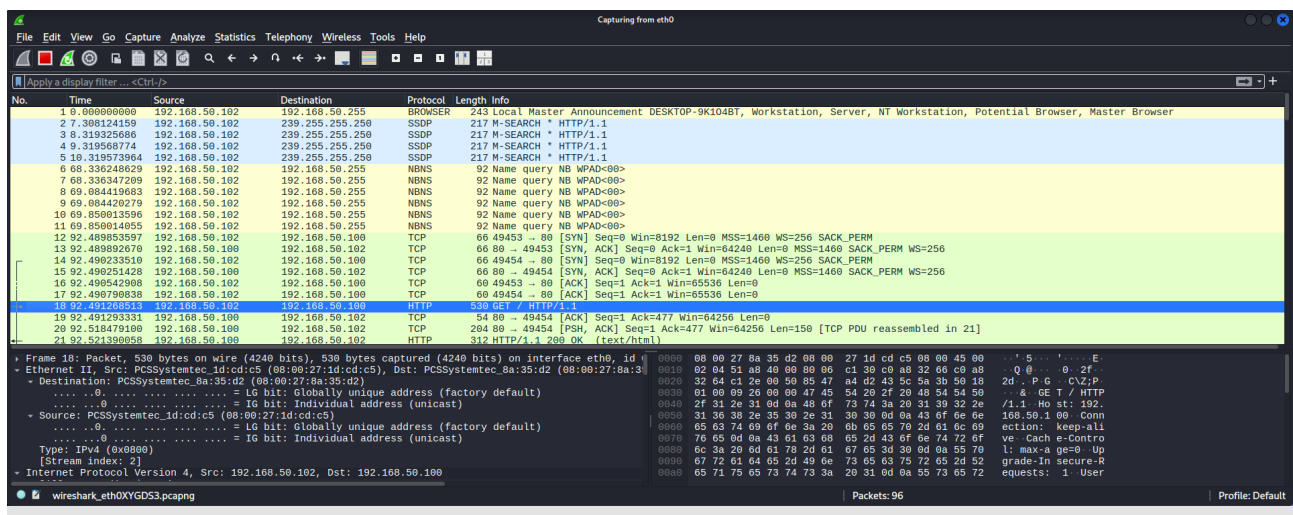
7. Sniffing delle comunicazioni (Attività facoltativa)

Durante la cattura del traffico di rete Wireshark ha permesso di osservare tutte le comunicazioni che transitavano sulla rete virtuale.

Oltre ai pacchetti HTTP sono stati rilevati anche protocolli come TCP, ARP e NBNS, utilizzati rispettivamente per la gestione della connessione, la risoluzione degli indirizzi MAC e alcuni servizi di rete di Windows.

L'analisi del traffico mi ha permesso di seguire l'intera comunicazione tra il client Windows e il server InetSim, osservando le richieste inviate e le relative risposte ricevute.

Figura 15 – Sniffing del traffico di rete



Conclusioni

L'esercitazione si è conclusa con successo. Dopo aver verificato la configurazione del firewall di Windows, è stato possibile comunicare correttamente tra le due macchine tramite il comando “ping”. Successivamente ho configurato e avviato InetSim, simulando diversi servizi di rete, tra cui il server HTTP. Infine, utilizzando Wireshark, ho osservato il traffico generato durante la comunicazione, analizzando i pacchetti HTTP, TCP e ARP e verificando il corretto funzionamento dei servizi simulati.

L'esercitazione mi ha permesso di comprendere meglio il funzionamento del firewall di Windows, della simulazione dei servizi di rete tramite InetSim e dell'analisi del traffico con Wireshark.